



Parul[®]
University

Lab Manual

iOS Application Development
(Practical)

Practical 1: Hello SwiftUI App - SwiftUI is a declarative framework used to build UI using code. UI updates automatically when data changes.

```
import SwiftUI

@main
struct HelloApp: App {
    var body: some Scene {
        WindowGroup {
            ContentView()
        }
    }
}

struct ContentView: View {
    var body: some View {
        ZStack {
            Color.blue.opacity(0.2)
                .ignoresSafeArea()

            VStack(spacing: 20) {
                Image(systemName: "hand.wave.fill")
                    .font(.system(size: 60))
                    .foregroundColor(.blue)

                Text("Hello, iOS Students!")
                    .font(.largeTitle)
                    .fontWeight(.bold)

                Text("Welcome to SwiftUI Development")
                    .font(.title3)
                    .foregroundColor(.gray)
            }
        }
    }
}
```

Practical 2: Login UI (Complete Design) - SwiftUI provides TextField and SecureField for input handling.

```
import SwiftUI

struct ContentView: View {
    @State private var username = ""
    @State private var password = ""
    @State private var message = ""

    var body: some View {
        VStack(spacing: 20) {

            Text("Login")
                .font(.largeTitle)
                .fontWeight(.bold)
```

```
TextField("Enter Username", text: $username)
    .padding()
    .background(Color.gray.opacity(0.2))
    .cornerRadius(10)

SecureField("Enter Password", text: $password)
    .padding()
    .background(Color.gray.opacity(0.2))
    .cornerRadius(10)

Button(action: validateLogin) {
    Text("Login")
        .frame(maxWidth: .infinity)
        .padding()
        .background(Color.blue)
        .foregroundColor(.white)
        .cornerRadius(10)
}

Text(message)
    .foregroundColor(.red)
}
.padding()
}

func validateLogin() {
    if username == "admin" && password == "1234" {
        message = "Login Successful ✓"
    } else {
        message = "Invalid Credentials ✗"
    }
}
}
```

Practical 3: Email Validation Form - Create a form that validates email format and password length before enabling login.

```
import SwiftUI

struct ContentView: View {
    @State private var email = ""
    @State private var password = ""
    @State private var message = ""

    var body: some View {
        VStack(spacing: 20) {

            TextField("Email", text: $email)
                .padding()
                .background(Color.gray.opacity(0.2))
                .cornerRadius(10)

            SecureField("Password", text: $password)
                .padding()
                .background(Color.gray.opacity(0.2))
                .cornerRadius(10)


```

```
Button("Submit") {
    if isValidEmail(email) && password.count >= 6 {
        message = "Valid Input ✓"
    } else {
        message = "Invalid Input ✗"
    }
}

Text(message)
}
.padding()
}

func isValidEmail(_ email: String) -> Bool {
    return email.contains("@") && email.contains(".")
}
}
```

Practical 4: Theme Color Changer (Full UI) - Create an app that changes background color when user taps different buttons.

```
import SwiftUI

struct ContentView: View {
    @State private var bgColor = Color.white

    var body: some View {
        VStack(spacing: 20) {
            Text("Choose Theme Color")
                .font(.title)

            HStack {
                Button("Red") { bgColor = .red }
                Button("Green") { bgColor = .green }
                Button("Blue") { bgColor = .blue }
            }

            .padding()
        }
        .frame(maxWidth: .infinity, maxHeight: .infinity)
        .background(bgColor.opacity(0.3))
    }
}
```

Practical 5: Counter App with Timer (Full UI) - Develop a counter that increments automatically with Start, Pause, and Reset controls.

```
import SwiftUI

struct ContentView: View {
    @State private var count = 0
    @State private var timer: Timer?

    var body: some View {
        VStack(spacing: 20) {

            Text("Counter")
```

```
.font(.largeTitle)

Text("\(count)")
.font(.system(size: 50))

HStack {
    Button("Start") {
        startTimer()
    }

    Button("Stop") {
        timer?.invalidate()
    }

    Button("Reset") {
        count = 0
    }
}

func startTimer() {
    timer = Timer.scheduledTimer(withTimeInterval: 1, repeats: true) { _ in
        count += 1
    }
}
```

Practical 6: Loading Screen (Progress Bar) - Create a loading screen that shows progress from 0% to 100% with status messages.

```
import SwiftUI

struct ContentView: View {
    @State private var progress = 0.0

    var body: some View {
        VStack(spacing: 20) {

            Text("Loading...")
                .font(.title)

            ProgressView(value: progress)

            Button("Start Loading") {
                startLoading()
            }

        }
        .padding()
    }

    func startLoading() {
        progress = 0.0

        Timer.scheduledTimer(withTimeInterval: 0.3, repeats: true) { timer in
            progress += 0.05
        }
    }
}
```

```
        if progress >= 1.0 {
            timer.invalidate()
        }
    }
}
```

Practical 7: Reminder App (Notification) - Allow user to enter a message and display a notification after a few seconds.

```
import SwiftUI
import UserNotifications

struct ContentView: View {

    var body: some View {
        VStack(spacing: 20) {

            Button("Set Reminder") {
                requestPermission()
                scheduleNotification()
            }

        }
    }

    func requestPermission() {
        UNUserNotificationCenter.current()
            .requestAuthorization(options: [.alert, .sound, .badge]) { _ , in }
    }

    func scheduleNotification() {
        let content = UNMutableNotificationContent()
        content.title = "Reminder"
        content.body = "This is your scheduled notification!"

        let trigger = UNTimeIntervalNotificationTrigger(timeInterval: 5, repeats: false)

        let request = UNNotificationRequest(identifier: UUID().uuidString, content: content, trigger: trigger)

        UNUserNotificationCenter.current().add(request)
    }
}
```

Practical 8: Prime Number Finder (UI Included) - Create an app to calculate prime numbers between a given range asynchronously.

```
import SwiftUI

struct ContentView: View {
    @State private var number = ""
    @State private var result = ""

    var body: some View {
        VStack(spacing: 20) {

            TextField("Enter Number", text: $number)
                .keyboardType(.numberPad)
```



Practical 9: Contacts Viewer - Display a list of contacts (name and number) using a scrollable list.

Page 7 of 7

```
    }  
  }  
}  
}.navigationTitle("Contacts")  
}  
}
```

Practical 10: Dynamic Cards Generator - Generate multiple cards dynamically based on user input.

```
import SwiftUI  
  
struct ContentView: View {  
  
    @State private var count = 0  
  
    var body: some View {  
        VStack {  
  
            Stepper("Number of Cards: \(count)", value: $count, in: 0...10)  
                .padding()  
  
            ScrollView {  
                VStack(spacing: 15) {  
                    ForEach(0..  
count, id: \.self) { i in  
                        VStack {  
                            Text("Card \(i+1)")  
                                .font(.headline)  
                                .foregroundColor(.white)  
                        }  
                        .frame(maxWidth: .infinity, minHeight: 100)  
                        .background(Color.blue)  
                        .cornerRadius(15)  
                        .padding(.horizontal)  
                    }  
                }  
            }  
        }  
    }  
}
```

Practical 11: Dialer App - Simulate a phone dialer that shows a calling message when user enters a number.

```
import SwiftUI  
  
struct ContentView: View {  
  
    @State private var number = ""  
    @State private var message = ""  
  
    let buttons = [  
        ["1", "2", "3"],  
        ["4", "5", "6"],  
        ["7", "8", "9"],  
        ["*", "0", "#"]  
    ]  
}
```



```
var body: some View {
    VStack(spacing: 20) {

        Text(number)
            .font(.largeTitle)
            .padding()

        ForEach(buttons, id: \.self) { row in
            HStack {
                ForEach(row, id: \.self) { item in
                    Button(action: {
                        number.append(item)
                    }) {
                        Text(item)
                            .frame(width: 70, height: 70)
                            .background(Color.gray.opacity(0.3))
                            .cornerRadius(35)
                            .font(.title)
                    }
                }
            }
        }

        Button("Call") {
            message = "Calling \(number)..."
        }
            .padding()
            .background(Color.green)
            .foregroundColor(.white)
            .cornerRadius(10)

        Text(message)
    }
}
```

Practical 12: Messaging Mock App - Create a simple chat interface where user can send and display messages.

```
import SwiftUI

struct ContentView: View {

    @State private var message = ""
    @State private var messages: [String] = []

    var body: some View {
        VStack {

            ScrollView {
                ForEach(messages, id: \.self) { msg in
                    HStack {
                        Text(msg)
                            .padding()
                            .background(Color.blue.opacity(0.2))
                            .cornerRadius(10)
                        Spacer()
                    }
                }
            }
        }
    }
}
```

```
    }  
    .padding(5)  
  }  
}  
  
HStack {  
  TextField("Type message...", text: $message)  
    .textFieldStyle(RoundedBorderTextFieldStyle())  
  
  Button("Send") {  
    if !message.isEmpty {  
      messages.append(message)  
      message = ""  
    }  
  }  
}  
  .padding()  
}
```

Practical 13: Notes App (File Storage) - Create a notes app that saves and retrieves text data from local storage.

```
import SwiftUI  
  
struct ContentView: View {  
  @State private var note = ""  
  
  var body: some View {  
    VStack(spacing: 20) {  
      TextEditor(text: $note)  
        .border(Color.gray)  
  
      HStack {  
        Button("Save") {  
          saveNote()  
        }  
  
        Button("Load") {  
          loadNote()  
        }  
      }  
    }  
    .padding()  
  }  
  
  func getFileURL() -> URL {  
    let path = FileManager.default.urls(for: .documentDirectory, in: .userDomainMask)[0]  
    return path.appendingPathComponent("note.txt")  
  }  
  
  func saveNote() {  
    try? note.write(to: getFileURL(), atomically: true, encoding: .utf8)  
  }  
}
```

```
func loadNote() {  
    if let data = try? String(contentsOf: getFileURL()) {  
        note = data  
    }  
}
```

Practical 14: Dark Mode Toggle (Persistent) - Implement a toggle switch to change between light and dark themes and save preference.

```
import SwiftUI  
  
struct ContentView: View {  
  
    @AppStorage("isDarkMode") private var isDarkMode = false  
  
    var body: some View {  
        VStack(spacing: 20) {  
  
            Text("Dark Mode Setting")  
                .font(.title)  
  
            Toggle("Enable Dark Mode", isOn: $isDarkMode)  
                .padding()  
  
        }  
        .preferredColorScheme(isDarkMode ? .dark : .light)  
    }  
}
```

Practical 15: Student Record App (CRUD) - Create a database-based app to perform CRUD operations on student data.

```
import SwiftUI  
  
struct Student: Identifiable {  
    let id = UUID()  
    var name: String  
    var age: String  
}  
  
struct ContentView: View {  
  
    @State private var students: [Student] = []  
    @State private var name = ""  
    @State private var age = ""  
  
    var body: some View {  
        VStack {  
  
            TextField("Enter Name", text: $name)  
                .textFieldStyle(RoundedBorderTextFieldStyle())  
  
            TextField("Enter Age", text: $age)  
                .textFieldStyle(RoundedBorderTextFieldStyle())  
  
            Button("Add Student") {
```

```
let student = Student(name: name, age: age)
students.append(student)
name = ""
age = ""
}

List {
  ForEach(students) { student in
    VStack(alignment: .leading) {
      Text(student.name)
      Text("Age: \(student.age)")
        .foregroundColor(.gray)
    }
  }
  .onDelete { indexSet in
    students.remove(atOffsets: indexSet)
  }
}
.padding()
}
```